HACKATHON BRIEF

GoTrain **Challenge**

Build an AI agent that builds geospatial models on its own · AUB ACM × RASID 48-hour hackathon

PREPARED FOR
AUB ACM Club · Hackathon teams

PREPARED BY
RASID · Geospatial Intelligence

DATE
31 October – 1 November 2026

REFERENCE / VERSION
GoTrain Challenge · v1.0

/ rasid.ai

Participant brief · Shared with all registered teams of the AUB ACM × RASID hackathon. Questions? Find the RASID team on site.

Contents

Why GoTrain 3
The challenge 3
Pick your problems 3
Build your minimal agent 4
Now the real work starts 4
What you submit 5
Resources 5
Extra points 6
How we recommend you start 6
How we'll judge 6
The prize 6
Further reading 7

Why GoTrain

Earth observation models are still built by hand. Every new sensor, region or task means an expert sits down, picks a backbone, designs a training recipe and tunes it against a benchmark. Over and over again.

Your job for the next 48 hours is to build an AI agent that does this automatically. You give it a geospatial dataset, and it explores the data, picks an approach, trains a model, checks its score and iterates. On its own.

> AT A GLANCE
>
> **The mission:** build GoTrain, an AI agent that trains geospatial models without a human in the loop.
>
> **The playground:** 4 problems of your choice from GEO-Bench (2 classification + 2 segmentation).
>
> **The tools:** Strands for the agent, Claude Sonnet on Amazon Bedrock, and your own GPU instance on Amazon SageMaker, all provided by us.
>
> **The prize:** an internship at RASID to take GoTrain to production inside GoPilot.

The challenge

Pick your problems

We're giving you 12 machine learning problems from GEO-Bench. Pick any **2 classification** and any **2 segmentation** problems.

- Our advice: start light. **m-eurosat** (276 MB) and **m-pv4ger-seg** (300 MB) are the smallest ones (highlighted below).
- Don't download the whole benchmark (it's about 17.5 GB). Grab only the tasks you picked from Hugging Face.

PROBLEM	WHAT IT PREDICTS	TRAIN / VAL / TEST	INPUT	SIZE
m-bigearthnet	Multi-label land cover (43)	20,000 / 1,000 / 1,000	Sentinel-2 · 12 bands · 10 m	6.56 GB
m-so2sat	Local climate zones (17)	19,992 / 986 / 986	Sentinel-1 + 2 · 18 bands · 10 m	964 MB
m-brick-kiln	Brick kiln yes / no	15,063 / 999 / 999	Sentinel-2 · 13 bands · 10 m	1.58 GB
m-forestnet	Deforestation driver (12)	6,464 / 989 / 993	Landsat-8 · 6 bands · 15 m	1.92 GB

PROBLEM	WHAT IT PREDICTS	TRAIN / VAL / TEST	INPUT	SIZE
m-eurosat	Land cover (10)	2,000 / 1,000 / 1,000	Sentinel-2 · 13 bands · 10 m	276 MB
m-pv4ger	Solar panel yes / no	11,814 / 999 / 999	Aerial RGB · 0.1 m	1.03 GB

Table 1 — Classification problems (one label per image).

PROBLEM	WHAT IT SEGMENTS	TRAIN / VAL / TEST	INPUT	SIZE
m-pv4ger-seg	Solar panels	3,000 / 403 / 403	Aerial RGB · 0.1 m	300 MB
m-chesapeake	Land cover (7)	3,000 / 1,000 / 1,000	Aerial RGBN · 1 m	1.25 GB
m-cashew-plant	Cashew plantations (7)	1,350 / 400 / 50	Sentinel-2 · 13 bands · 10 m	1.33 GB
m-SA-crop-type	Crop types (10)	3,000 / 1,000 / 1,000	Sentinel-2 · 13 bands · 10 m	1.43 GB
m-nz-cattle	Cattle	524 / 66 / 65	Aerial RGB · 0.1 m	327 MB
m-NeonTree	Tree crowns	270 / 94 / 93	RGB + hyperspectral + elevation · 0.1 m	601 MB

Table 2 — Segmentation problems (one label per pixel).

Build your minimal agent

Go to Strands and build your minimal agent. You can check all the sample agents Strands provides here.

Your agent runs its ReAct loop with Claude Sonnet on Amazon Bedrock, for free. No API key needed: your SageMaker instance already has access (see Resources for a quick test).

Now the real work starts

You have a minimal agent, and your task is to grow it into something that can actually train models on these datasets. By improving the agent we mean things like:

- Improving the system prompt
- Building tools to explore and prepare the dataset
- Building tools to upload the dataset to the cloud and train on a GPU
- Building tools to score the model and iterate on earlier versions
- …and honestly, get as creative as you want

The idea: your agent explores the data, picks an approach, trains, checks its score, and iterates. On its own.

> **ONE RULE WE'RE STRICT ABOUT**
> Your agent tunes and iterates on the **validation split only**. The **test split is touched once**, at the very end, with your final model.
>
> Keep your agent's logs. We want to see how every score was produced.

What you submit

1. Your GitHub repo with the agent you built
2. Your test-split scores on your 4 problems, plus the agent logs
3. A presentation explaining what you built

Resources

Each team gets its own **Amazon SageMaker** instance with a GPU, and that's where you'll work for the whole hackathon. SageMaker is the only place your agent can talk to the Anthropic model on Amazon Bedrock, so build and run everything there.

- **How you get in:** we'll send your team a link that opens JupyterLab in your browser. Clone your agent there, run `pip install geobench strands-agents`, and run your agent loop.
- **Good to know about your link:** open it within 5 minutes of receiving it, then bookmark the JupyterLab page. A session lasts up to 12 hours; when it ends, just ask us for a new link. Your files and running training stay on the instance, so nothing is lost.
- **No API keys needed:** your instance already has access to Bedrock. Use the model `eu.anthropic.claude-sonnet-5-5` in the region `eu-central-1`.

Open a notebook and try this first. If your agent answers, you're ready to go:

```
from strands import Agent
from strands.models import BedrockModel

model = BedrockModel(
    model_id="eu.anthropic.claude-sonnet-5-5",
    region_name="eu-central-1",
)
agent = Agent(model=model)
agent("Hello, what can you do?")
```

And don't worry, our team will be there to help you with infrastructure if you get stuck.

Extra points

- In a real production system, datasets and model versions don't just sit in one notebook folder. If your agent has tools that upload and download datasets and models to S3 buckets or a file system, and keeps track of model versions, you get extra points.
- Didn't get to build the production setup? No problem. Show us in your presentation how you would have built it if you had the time.
- Don't limit yourself to 4 datasets. If your agent is already training models well, go ahead and run it on more.

How we recommend you start

1. **Hop on Strands first** and understand what an AI agent is, what a ReAct loop is, what MCP is, and what a tool call is. This way you know what you're building. Feel free to jump on GoPilot, our own AI agent, and chat with it a little.
2. **Train a small model yourself.** Pick one dataset and see how it goes. You can't expect an agent to train a model if you don't know how to do it yourself 👀
3. **Turn your work into tools.** Look at what you did and ask: what tools would the agent need to replicate and generalize this? Then start building them.

How to split your time: spend about 75% of day one on steps 1 and 2. Use the rest of day one and all of day two to build GoTrain.

How we'll judge

When the hackathon ends, we'll hear your presentation and look at:

- How you approached the problem
- Whether you came up with a nice or creative workflow for training models
- Your test-split scores and the logs behind them
- Whether your repo is clean enough to plug straight into GoPilot

The prize

> FOR THE WINNERS
>
> An **internship at RASID**, where you'll keep developing this agent, take it to production, and ship it as an MCP server for GoPilot, RASID's geospatial AI agent.
>
> **Good luck, and have fun!**

Further reading

Totally optional, but if you want to see how others are tackling the same idea, these are worth a look:

- RSI4EO: a coding agent builds state-of-the-art earth observation models given only the data and a score.
- karpathy/autoresearch: the simplest version of the idea. The agent edits a training script, trains for 5 minutes, and keeps the change only if the score improves. A 10-minute read.
- Autoresearch for solar panel segmentation: the same loop on a geospatial task, with small models and a single GPU. The closest thing to what you'll build.
- AIDE: tree search over solution scripts. Useful if you want your agent to try several approaches instead of one at a time.
- GEO-Bench: the benchmark paper, with baseline scores for every task, so you know what a good score looks like.